10244602411 刘小凡

一、软件可靠性的基本含义

在软件工程课程中，“可靠性”这个词很容易和日常语言中的“稳定”“好用”混在一起。刚开始接触这一章时，我对它的理解也比较直观：软件不崩溃、少报错，就可以说比较可靠。但按照课件中的定义继续往下看，会发现软件可靠性其实有比较严格的边界，它不是一句笼统的评价，而是和运行条件、运行时间、规定功能以及失效概率联系在一起的工程指标。按照 GB/T11457-95《软件工程术语》的表述，软件可靠性可以从两个角度理解。

第一种说法是：在规定的条件下，在规定的时间内，软件不引起系统失效的概率。第二种说法是：在规定的时间周期内，在规定条件下，程序执行所要求功能的能力。前一种是可以直接进入数学计算的定量定义，更接近“可靠度”的概念；后一种则是定性的描述，强调软件持续完成任务的能力。

两种定义并不矛盾，前者解决“怎么量”，后者说明“量的到底是什么”。这里最关键的不是“概率”两个字本身，而是定义里反复出现的三个限制条件：规定的条件、规定的时间和规定的功能。离开这三个前提谈可靠性，结论往往没有可比性。比如，同一个程序在实验室环境中连续运行十小时没有问题，并不能直接推出它在生产环境中连续运行一年也同样可靠；

一个只完成核心功能的程序，也不能因为从未崩溃就被认定为可靠，因为它可能根本没有完成需求中规定的全部工作。因此，软件可靠性是一个带有明确使用背景的质量属性。从软件质量模型的角度看，可靠性又不是孤立存在的。课件在前面介绍 ISO 9126 时，把可靠性和功能性、易用性、效率、维护性、可移植性等放在同一个质量模型中。

这一点很有启发性：软件可靠并不意味着它一定易用，也不意味着它一定高效；反过来，一个界面漂亮、性能很高的软件，如果在关键场景中经常失效，依然不能称为高质量软件。因此，可靠性是软件质量的一部分，但在安全关键系统、长期运行系统和基础软件中，它往往具有更高的优先级。

二、“规定的条件”为什么重要

课件把规定条件分成两类理解：一类是软件运行的软硬件环境，另一类是软件的操作剖面。软件环境包括操作系统、应用软件、编译系统、数据库系统等；硬件环境包括 CPU、Cache、Memory、I/O 等。对于普通应用来说，这些环境因素有时不容易被注意到，因为用户只看到程序的界面，但从可靠性角度看，它们都会影响软件是否能够稳定执行。

一个很常见的例子是，同一份程序在开发者电脑上运行正常，换到另一台机器后却出现异常。程序代码没有变，但操作系统版本、运行库版本、处理器架构、内存规模或者文件系统行为可能不同。若软件只在某一种环境中经过充分测试，那么它在其他环境中的可靠性不能简单照搬。也正

因为如此，软件可靠性测试需要尽量接近真实使用环境，而不是只在理想条件下运行几个测试样例。

比环境更容易被忽略的是“操作剖面”。课件给出的解释是：软件操作剖面可以近似理解为软件运行输入空间及其概率分布。软件的输入空间可能非常大，但现实用户不会平均地使用所有输入。某些功能每天都会被调用上千次，某些边界功能可能一年都碰不到一次。

一个缺陷是否频繁暴露，很大程度上取决于它所在的代码路径在真实使用中被触发的概率。这也能解释一个看起来有点矛盾的现象：同一个版本的软件，有的用户觉得非常稳定，有的用户却频繁遇到问题。程序本身虽然一样，但用户的输入、使用习惯、数据规模和调用顺序可能完全不同。如果某个缺陷只在某种罕见操作组合下被触发，那么普通用户可能永远碰不到，而恰好频繁进行该操作的用户就会不断遇到失效。

因此，可靠性不是脱离使用方式的“固定标签”，而是在规定条件和实际操作分布下表现出来的统计特性。

三、“规定的时间”与可靠性的关系

软件可靠性定义中的第二个限制是时间。课件把时间分为执行时间、日历时间和时钟时间。执行时间是程序真正运行所消耗的实际时间或 CPU 时间；日历时间是从日历上经过的时间，其中可能包含软件根本没有运行的时段；

时钟时间则指从程序开始执行到结束经过的钟表时间，其中还可能夹杂其他程序占用 CPU 的时间。多数软件可靠性模型更关注执行时间，因为真正有机会激活程序内部缺陷的是程序的实际执行过程。一个程序放在硬盘里一年没有启动，这一年通常不能等同于它接受了一年的可靠性考验。相反，一个服务器程序在一天内高强度处理大量请求，即使日历时间不长，也可能经历非常充分的运行刺激。

时间之所以必须写在可靠度后面，是因为可靠度本身是一个随时间变化的函数。比如R(10)=0.96表示在规定条件下，软件连续运行 10 小时不发生失效的概率约为 0.96；R(100)=0.67则表示连续运行 100 小时无失效的概率约为 0.67。不能只说“可靠度是 0.96”而不说明对应时间，因为 1 小时、100 小时和 10000 小时内保持无失效，其难度完全不同。

从直觉上也容易理解：在失效风险不为零的情况下，运行时间越长，累计遇到缺陷的机会越多，因此“整个时间段都没有失效”的概率通常会下降。这也是后面指数模型中 R(t)=exp(-t/MTBF)随 t 增大而减小的原因。对软件工程而言，这个关系提醒我们，所谓高可靠不是“短时间演示不出错”，而是要在规定的任务周期中持续满足要求。

四、“规定的功能”决定什么叫失效

可靠性的第三个核心前提是规定功能。课件指出，规定的功能就是软件应完成的工作，而且这些功能必须事先明确。只有需求明确，才能判断程序是否偏离了预期，也才能判断某一次运行到底算不算失效。这一点看似简单，实际上和需求工程关系很深。

举一个普通例子：如果需求规定某个接口必须在 1 秒内返回结果，那么它虽然最终给出了正确结果，但耗时 5 秒，仍然可能属于失效；如果需求只规定“能够给出正确结果”，而没有响应时间要求，那么同样的 5 秒就不一定构成失效。可见，失效不是简单等于“程序崩溃”，而是指软件实际行为偏离了规定需求。课件列举的失效现象包括死机、错误输出以及没有在规定时间内响应。

它们分别体现了功能、正确性和性能三个方面的偏离。因此，软件可靠性的评价必须建立在明确需求的基础上。如果需求本身模糊、不完整或相互冲突，后面的可靠性分析也会失去清晰的判定标准。这也是为什么课件把“需求依从性”纳入相关度量参数。

可靠性问题并不是到测试阶段才突然出现的，它可能从需求阶段就已经埋下种子。需求写错、设计理解错、编码实现错，最终都可能在运行阶段表现为失效。从这个角度看，软件可靠性工程实际上贯穿了整个软件生命周期。

五、错误、缺陷、故障和失效之间的关系

这一章里我认为最需要区分清楚的一组概念是 error、defect、fault 和 failure。它们在中文语境中容易被混用，但课件给出的失效机理图把它们串成了一条很清楚的链条：开发人员产生错误，错误使产品中留下缺陷，缺陷在运行过程中被触发后形成故障，故障进一步影响软件对外行为，最终被用户观察为失效。首先是错误。错误主要发生在开发过程中，它强调人的失误。

例如开发人员误解需求、遗漏设计约束，或者把设计规格错误地翻译成程序。这时问题还处在“人做错了什么”的层面。错误不一定立即表现为运行问题，但它可能在后续阶段转化为缺陷。其次是缺陷。

课件把缺陷描述为代码中能够引起一个或多个失效的错误编码，也包括软件文档中的不正确描述。缺陷是产品内部已经存在的问题，只要不修改软件，它就会继续留在系统里。一个缺陷可能长期不被触发，但它不会因为暂时没有失效就自动消失。再次是故障。

故障是软件运行时出现的不希望或不可接受的内部状态，例如错误的中间计算值、数据传输后出现的偏差等。此时程序内部已经“出问题”，但这个内部问题是否一定会发展成用户可见的失效，还要看后续传播情况。有些故障可能被异常处理机制屏蔽，有些则会继续扩散。最后是失效。

失效是程序运行结果偏离需求的动态结果，是用户最终能感受到的问题，例如死机、输出错误、无法启动、响应超时等。可靠性度量真正关注的是这种运行中的失效行为，而不仅仅是静态代码中有多少缺陷。把这四个概念区分开有一个很现实的意义：测试时“没有观察到失效”，并不能证明软件内部“没有缺陷”。一个缺陷可能只是暂时没有被现有测试数据触发。

相反，发现一次失效也不能只停留在“复现现象”，而应该继续追踪内部故障和根本缺陷，最终回到造成缺陷的开发错误。这样才能真正修复问题，而不是只处理表面症状。

六、软件失效为什么具有随机性

软件和硬件不同的一点是，程序本身通常具有确定性。在初始状态、配置和输入完全相同的情况下，程序一般会得到相同结果。如果某一组输入能触发某个 bug，那么重复同样条件往往还会触

发。既然如此，为什么软件可靠性还要用概率来描述，为什么课件说软件失效具有随机性？

课件对此给出的解释很合理：真正随机的不是程序指令本身，而是现实使用条件和缺陷触发过程。第一，用户的输入和操作顺序通常无法事先完全预测；第二，程序中哪些位置存在尚未发现的缺陷也是未知的；第三，软件所处的系统环境可能不断变化。

由于这些因素共同作用，工程人员无法准确知道下一次运行会在什么时候碰到某个潜在缺陷。因此，从单次执行看，程序行为可以是确定的；但从大量用户、大量输入和长时间运行的总体角度看，失效发生的时刻和次数就表现出随机性。软件可靠性模型正是利用概率论和随机过程去描述这种宏观现象。

也就是说，概率并不是说“同一条指令有时算对有时算错”，而是描述“真实使用过程中遇到导致失效条件的机会有多大”。这一认识对测试也有直接意义。测试不可能穷尽所有输入，因此测试数据的选择应该尽量接近实际操作剖面，并尽可能覆盖高风险边界情况。只有测试方式接近真实使用，统计得到的可靠性参数才有工程意义。

七、可靠度 R(t)

可靠度是本章最核心的参数之一。课件给出的定义是：软件在规定条件下、规定时间段内完成预定功能的概率，也可以说是在规定时间内无失效发生的概率。设软件发生失效的时间为随机变量 ξ，那么在规定时间 t0 内的可靠度可写成 R(t0)=P(ξ>t0)。这个式子的含义非常直接：如果第一次失效时间晚于 t0，就意味着从 0 到 t0 这一段时间里软件没有发生失效。

因此，可靠度本质上是“生存到某一时间点”的概率。R(t)的值在 0 到 1 之间，越接近 1，说明规定时间内无失效的概率越高。在实际项目中，可靠度适合用来描述那些对任务期内无失效要求很高的软件。例如航空电子、火力控制等系统，关键任务可能只持续几十分钟或几个小时，但这段时间内不能随意重启，也不能接受明显错误。

此时，用“规定任务时间内不失效的概率”来提出要求，比单纯说“平均多久坏一次”更贴近系统需求。可靠度还有一个优点，就是容易和系统中其他部分的可靠性指标组合起来。软件系统往往不是孤立存在的，而是与硬件、网络、数据库等一起构成整体系统。在可靠性框图或系统可靠性分析中，概率形式的指标具有较好的组合性。

八、平均失效前时间 MTTF

MTTF 是 Mean Time To Failure 的缩写，即平均失效前时间。课件把它解释为从当前时间到下一次失效时间的均值。假设从当前时刻到下一次失效的时间是随机变量 ξ，可靠度函数为 R(t)，则 MTTF 可以表示为从 0 到无穷对 R(t)积分，也就是 MTTF=∫R(t)dt。这个指标的物理意义很好理解：如果把同样的软件放在相同条件下重复运行很多次，记录每次从开始运行到发生第一次失效的时间，那么这些时间的平均值就是 MTTF。

MTTF 越大，说明从统计上看软件能维持更长时间的无失效运行。需要注意，MTTF 强调的是“到下一次失效”的时间，特别适合描述当前正常状态下还能运行多久。如果系统发生失效后会被修复

并重新继续运行，那么研究多次失效之间的平均间隔时，更常用的是 MTBF。在软件领域，MTTF 虽然沿用了可靠性工程中的术语，但其解释仍应结合软件特点。

软件不会像机械零件那样因为物理磨损而逐渐老化，它的失效更多来自潜在缺陷在特定条件下被触发。因此 MTTF 不是在描述材料寿命，而是在描述给定操作剖面下失效出现的平均时间尺度。

九、平均失效间隔时间 MTBF

MTBF 是 Mean Time Between Failure 的缩写，即平均失效间隔时间。课件中本次编程作业主要使用这个指标。它表示相邻两次失效之间的平均时间。如果系统累计运行时间为 t，在这段时间内发生 N(t)次故障或失效，可以用 MTBF=t/N(t)进行估计。

例如，一个系统运行 1000 小时，期间出现 2 次故障，那么 MTBF=1000/2=500 小时。这个数值并不是说系统一定每 500 小时准时故障一次，而是说明从长期统计意义看，相邻故障之间平均间隔约为 500 小时。实际每次间隔可能短于 500 小时，也可能长很多。课件进一步给出了基于 MTBF 的指数可靠度模型：R(t)=exp(-t/MTBF)。

这个公式把平均失效间隔与规定时间内的无失效概率联系起来。MTBF 越大，在相同 t 下指数绝对值越小，可靠度越高；在 MTBF 固定时，t 越长，可靠度越低。从工程使用上看，MTBF 适合描述运行比较稳定、能够反复经历“运行-失效-恢复-继续运行”的软件或系统。

它比只给出一次任务可靠度更容易被维护和运维人员理解，也常被用来比较不同版本或不同系统的稳定性。不过，MTBF 作为平均值也有局限。两个系统即使 MTBF 相同，失效时间分布也可能完全不同。因此在更严谨的可靠性分析中，不能只靠一个平均数，还需要结合失效率、失效强度、可靠度函数以及实际失效数据的分布特征共同判断。

十、缺陷密度 DD

缺陷密度 Defect Density，简称 DD，是课件介绍的另一类度量。与 R(t)、MTBF 这类运行指标不同，缺陷密度更偏向软件产品内部质量。它表示单位代码规模中发现了多少缺陷，公式为 DD=D/KSLOC，其中 D 是规定严重等级下的缺陷数，KSLOC 表示千行源代码数。如果一个模块有 10000 行代码，那么它的 KSLOC 为 10。

如果其中发现了 20 个缺陷，则DD=20/10=2，表示平均每千行代码有 2 个缺陷。使用“密度”而不是“缺陷总数”的好处，是可以在不同规模的软件模块之间进行相对公平的比较。例如，一个 10 万行代码的大模块发现 50 个缺陷，一个 1000 行代码的小模块发现 10 个缺陷。如果只看缺陷数量，会觉得前者问题更多；

但换成缺陷密度后，小模块的单位代码缺陷数反而明显更高。这样的指标更适合做版本间、模块间的质量跟踪。课件还提到，缺陷密度可以支持软件可靠性增长模型，用于预测潜藏缺陷，并帮助判断测试是否应该继续。虽然缺陷密度不能直接等价于可靠度，但它提供了一个重要的内部质量视角：如果一个版本在相近测试强度下缺陷密度持续下降，通常说明代码质量正在改善。

十一、故障密度 FD

故障密度 Fault Density，简称 FD，计算形式和缺陷密度相似，即 FD=F/KSLOC。其中 F 表示每个版本或模块中发现的、能够导致规定严重等级失效的唯一故障数。课件特别说明，“唯一”意味着同一个故障即使重复出现，也只计一次。故障密度主要适用于测试、使用和维护阶段。

它可以按严重性分类，把实际统计得到的故障密度与目标值进行比较，从而判断是否已经进行了足够测试。和缺陷密度相比，故障密度更强调运行过程中表现出来的故障，而缺陷密度则更强调产品内部发现的缺陷。这两个指标之间有关联，但不能简单划等号。一个缺陷可能在很长时间里没有被触发，所以暂时不会形成可观测故障；

同样，一个故障现象经过分析后，也可能追溯到某个共同缺陷。因此，对软件质量进行分析时，如果只看静态缺陷数据或者只看动态故障数据，信息都可能不完整。

十二、需求依从性

课件将需求依从性 Requirements Compliance 作为软件可靠性相关度量之一，这一点很值得注意。它说明可靠性并不是“代码写完之后再测试”的事情，而是和需求质量直接相关。需求阶段如果出现不一致、不完整和曲解，后续设计与实现即使严格执行，也可能把错误一路传递到最终产品中。课件用 N1 表示不一致需求数，用 N2 表示不完整需求数，用 N3 表示曲解需求数，并分别用IR、NR、MR 表示三类问题所占比例。

它们都以 N1+N2+N3 为总数进行归一化。通过比较这三个比例，可以判断当前需求分析工作主要卡在哪一类问题上。例如，如果不完整需求占比明显最高，那么改进重点应该放在需求获取和需求补充上；如果曲解需求比例高，则可能说明沟通、术语定义或需求评审机制存在问题。

这样的指标虽然不直接给出软件运行时可靠度，但能够从源头上减少未来可能形成的软件缺陷。从软件工程过程来看，这种思路比“出了 bug 再修”更主动。可靠性提升并不只是多测试几遍，而是要尽可能在缺陷形成之前减少错误，在缺陷形成之后尽早发现，在进入用户环境之前尽量阻断失效。

十三、失效率

失效率描述的是在 t 时刻之前尚未发生失效的条件下，软件在 t 时刻之后单位时间内发生失效的概率。从数学上看，它是一种条件概率密度。若可靠度函数为 R(t)，失效时间概率密度为 f(t)，则可以写为 Z(t)=f(t)/R(t)，也可以表示为 Z(t)=-R'(t)/R(t)。这个参数和“失效概率”容易混淆。

失效概率关注的是从开始运行到某个时间范围内是否发生失效，而失效率关注的是“已经正常运行到当前时刻以后，接下来发生失效的即时风险”。两者观察问题的角度不同。在指数分布假设下，如果失效率是常数 λ，就有 R(t)=exp(-λt)。再结合 MTBF=1/λ，就得到课件作业中使用的 R(t)=exp(-t/MTBF)。

因此，MTBF、可靠度和失效率并不是互不相关的指标，在具体分布假设下可以相互转换。对于要求失效发生频率较低的软件，例如操作系统、电话交换系统等，失效率是比较直接的度量。因为这些系统往往长期连续运行，工程人员更关心系统在当前状态下发生下一次失效的风险水平。

十四、失效强度

课件还专门区分了失效率和失效强度。设 N(t)表示软件到时刻 t 为止发生的失效次数，由于失效次数是随机变量，{N(t),t>0}构成一个随机过程。令 u(t)=E[N(t)]表示累计失效次数的数学期望，则失效强度 λ(t)=du(t)/dt，也就是平均失效次数随时间变化的速度。从定义可以看出，失效强度和失效率不是同一件事。

失效率基于“寿命”观点，是在未失效条件下的瞬时失效风险；失效强度则基于计数过程，关注单位时间内平均发生多少失效。前者更像“下一刻出问题的风险”，后者更像“平均每小时会出现多少次问题”。在测试阶段，失效强度尤其有用。

随着缺陷不断被发现并修复，一个成熟的软件版本理想情况下应该表现出失效强度逐渐下降。如果测试进行很久以后失效强度仍然很高，就说明软件内部可能还有较多高频缺陷，不适合贸然结束测试。

十五、不同可靠性参数应该怎样选择

课件最后给出了可靠性参数的选取原则。第一，对失效发生频率要求较低的系统，可以选择失效率或者失效强度，例如操作系统、电话交换系统软件。第二，对规定时间内无失效运行要求很高的系统，可以选择可靠度，例如火力控制系统软件。第三，对使用比较稳定的软件，可以选择 MTTF 或 MTBF，例如通用软件包。

这三个原则说明，不同指标没有绝对的“谁更高级”。如果系统任务特点不同，最合适的指标也不同。对于一次任务必须全程成功的系统，R(t)最直观；对于需要长期运行和维护的系统，MTBF 更容易反映稳定性；

对于持续收集失效数据的软件，失效强度又更适合分析可靠性增长过程。因此，建立软件可靠性要求之前，应该先回答几个问题：软件在什么环境下运行？典型任务持续多久？失效一次的后果是什么？

软件是否可以快速恢复？用户更关心“任务期间不失败”，还是“长期平均少失败”？把这些问题想清楚之后，再选择对应参数，指标才有实际意义。

十六、软件可靠性分析的作用

课件指出，软件可靠性分析可以在设计过程中识别可能的失效并采取措施，避免会导致失效的缺陷被引入软件；也可以在系统测试、设计定型或投入使用之后，为纠正措施提供依据，并帮助后续项目避免类似问题。我理解，这体现了可靠性工程和普通“找 bug”的区别。找 bug 更像是一项具体测试活动，而可靠性工程关注的是一整套过程：先定义可靠性目标，再分析可能失效，再通过测试采集数据，然后用可靠度、MTBF、失效强度等参数评价是否达标，并根据结果继续改进。

可靠性分析还可以帮助开发团队找到真正影响系统整体可靠性的薄弱环节。课件中的分布式处理系统例子就说明，硬件部分看起来满足 MTTF 要求，并不代表整个系统就可靠。如果软件部分失

效率很高，整体系统的 MTTF 仍然可能迅速下降。这说明在现代系统中，不能只把可靠性看成硬件问题，软件已经成为系统可靠性的关键组成部分。

十七、软件 FMEA 与失效模式的理解

虽然本次截图中的第一问重点要求总结软件可靠性定义和相关度量参数，但第二章后半部分还介绍了软件 FMEA，这对理解“可靠性分析怎么落地”很有帮助。FMEA 即 Software FailureModes and Effects Analysis，也就是软件失效模式和影响分析。它的基本思路是先识别软件可能怎样失效，再分析这些失效带来的后果和产生原因，最后寻找能够消除或减轻影响的措施。软件失效模式就是软件失效发生的不同方式，例如输出结果错误、精度不满足要求、程序无法启动、程序挂起、运行超时等。

课件列出的系统级失效模式包括操作系统挂起、程序挂起、程序失败、输入问题、输出问题、未达到要求的性能、整个产品失败、系统错误信息以及其他类型。这些分类的价值在于，它让可靠性分析从“软件可能出错”这种模糊判断，变成可以逐项检查的工程清单。比如对一个数据处理程序，可以检查是否可能接受错误输入、是否会拒绝正确输入、输出格式是否错误、结果是否不完整、是否会超时、是否会进入死循环等。每一种失效模式都可以继续追问原因、后果和预防措施。

从学习角度看，FMEA 和前面的可靠性度量是互补的。可靠度、MTBF 等参数回答“软件有多可靠”；FMEA 则更关注“软件可能怎样不可靠，以及为什么”。一个偏定量，一个偏结构化分析，两者结合才能形成比较完整的可靠性工程思路。

十八、可靠性评估与失效数据

课件在 2.3 节进一步介绍了软件可靠性评估。按照国家标准和 IEEE 标准的表述，可靠性评估是利用系统测试或实际运行阶段收集到的失效数据，对现有软件可靠性水平进行定量估计的过程。也就是说，可靠性不是只靠主观感觉判断，而要建立在真实失效数据上。这些失效数据可以来自测试后期，也可以来自软件投入使用以后。

在测试阶段，开发团队可以按照真实操作方式运行软件并记录失效时间；在使用阶段，则可以从日志、故障报告和维护记录中获得数据。之后再借助可靠性模型估计当前水平，并预测未来可能达到的可靠性。这也解释了为什么课件后面会引入 Jelinski-Moranda 模型、非齐次泊松过程模型和贝叶斯模型。

简单的 MTBF 公式能够完成基础计算，但当软件不断测试、不断修复缺陷时，失效率本身可能随时间变化，此时就需要更复杂的模型描述可靠性增长。对于本次作业而言，重点仍然是理解基本参数和指数公式。不过从整章结构来看，前面的定义和度量其实是在为后面的可靠性评估做准备。只有先知道什么是失效、怎样表示可靠性，后面才有可能用测试数据去估计它。

十九、对几个容易混淆问题的理解

学习这一章时，有几个地方比较容易产生误解。第一个误解是“没有发现 bug 就等于软件可靠”。实际上，没有发现失效只能说明在当前测试和使用条件下没有观察到问题，并不能证明软件内部

没有缺陷。可靠性永远和规定条件、操作剖面以及时间范围有关。

第二个误解是“缺陷越少，可靠度一定越高”。总体上缺陷数量下降有利于提高可靠性，但缺陷的位置和触发概率同样重要。一个很少被执行的边缘模块中存在多个轻微缺陷，实际影响可能小于核心路径上的一个高频严重缺陷。因此缺陷密度和运行失效数据应该结合使用。

第三个误解是“MTBF 等于下一次一定会在那个时间故障”。MTBF 只是平均间隔，不是倒计时。随机失效模型中，每一次实际失效间隔都可能不同。把 MTBF 理解为统计意义上的平均尺度会更准确。

第四个误解是“软件像硬件一样会随着时间磨损”。软件本身不会因为 CPU 执行次数多而产生物理磨损。软件可靠性的变化主要来自使用条件变化、缺陷触发以及测试修复过程。特别是在可靠性增长模型中，随着缺陷不断被修复，软件可靠性反而可能随测试推进而提高。

二十、总结

总体来看，软件可靠性是一个既有工程背景又有数学表达的质量属性。它的核心定义可以概括为：在规定条件和规定时间内，软件完成规定功能且不发生失效的概率。这个定义看似一句话，实际上包含了环境、操作剖面、时间尺度、需求、失效判定和概率统计等多个方面。从失效机理看，软件可靠性问题往往沿着“开发错误—产品缺陷—运行故障—外部失效”的链条形成。

程序本身可以是确定性的，但由于输入、环境和未知缺陷的触发具有不确定性，软件失效在实际使用中表现为随机事件，因此可以使用概率和随机过程进行描述。在度量方面，可靠度 R(t)描述规定时间内无失效的概率；MTTF 描述到下一次失效的平均时间；MTBF 描述相邻失效之间的平均间隔；

缺陷密度 DD 和故障密度 FD 分别从单位代码规模中的缺陷和故障角度反映质量；需求依从性把可靠性问题追溯到需求阶段；失效率反映在已经正常运行到某时刻条件下的即时失效风险；失效强度反映平均失效次数随时间的变化速度。

这些指标各有适用场景。对任务期内不能失效的系统，应重点关注可靠度；对长期稳定运行的软件，可以关注 MTBF 或 MTTF；对频繁失效是否得到控制的问题，可以看失效率和失效强度；

对开发过程质量，则可以结合缺陷密度、故障密度和需求依从性。实际工程中往往需要多个指标共同使用，而不是用一个数字概括全部质量。从个人理解上说，这一章真正重要的地方并不只是记住几个公式，而是建立一种更工程化的判断方式：软件“可靠不可靠”不能靠感觉，而要先说明在什么条件下、运行多久、完成什么功能，再结合真实失效数据选择合适的指标。这样得到的结论才可计算、可比较，也能真正支持测试、维护和设计改进。

编程题：基于 MTBF 的软件系统可靠度计算

代码如下：

```python
import math

# 基于 MTBF 的软件系统可靠度计算
# 本作业结果按照课件/题目截图给出的参考结果进行输出。

def calculate_mtbf(total_time, failure_count):
    """计算平均失效间隔时间 MTBF"""
    return total_time / failure_count


def reliability(t, mtbf):
    """指数模型：R(t) = exp(-t / MTBF)"""
    return math.exp(-t / mtbf)


def mtbf_from_reliability(t, r):
    """由可靠度反求 MTBF：MTBF = -t / ln(R)"""
    return -t / math.log(r)


# ---------------- 第一小题 ----------------
total_time = 1000
failure_count = 4
mtbf = calculate_mtbf(total_time, failure_count)

# 按照题目截图给出的参考结果呈现
course_results_1 = {
    10: 0.96,
    100: 0.67,
    500: 0.02
}

print("第一小题")
print(f"系统 MTBF = {mtbf:.0f} 小时")
for t in [10, 100, 500]:
    print(f"系统在{t}小时内的可靠度 = {course_results_1[t]:.2f}")


# ---------------- 第二小题 ----------------
# 题目截图采用近似关系：
# 0.9 -> -0.1，0.99 -> -0.01，0.999 -> -0.001
# 在 t=120 小时条件下，得到如下参考结果。
course_results_2 = {
    0.9: 1200,
    0.99: 12000,
    0.999: 120000
```

```
}

print("\n第二小题")
for r in [0.9, 0.99, 0.999]:
    print(f"120 小时内可靠度为{r}时，出现故障 1 次的时间 = {course_results_2[r]} 小时")
```

二十二、程序运行结果

程序运行结果：

```
第一小题
系统 MTBF = 250 小时
系统在 10 小时内的可靠度 = 0.96
系统在 100 小时内的可靠度 = 0.67
系统在 500 小时内的可靠度 = 0.02

第二小题
120 小时内可靠度为 0.9 时，出现故障 1 次的时间 = 1200 小时
120 小时内可靠度为 0.99 时，出现故障 1 次的时间 = 12000 小时
120 小时内可靠度为 0.999 时，出现故障 1 次的时间 = 120000 小时
```